SIMATS ENGINEERING COLLEGE

C PROGRAMMING — CO1 ASSIGNMENT 1

Design, Implement and Validate a Reliable Billing Program
for a Rural Electricity Co-operative

Tasks 1–4 | Total assignment: 50 marks

Name	AKSHAYA R S
Register Number	1924260209
Course Code	CSA0209B
GitHub Repository Link	https://github.com/akshaya12258-droid/CSA0209-C-PROGRAMMING-1924260209
Submission Date	27 October 2026

Contents

- Task 1 — Understand the Problem and Identify the Data (4 Marks)
- Task 2 — Choose the Data Types (4 Marks)
- Task 3 — Define the Constants
- Task 4 — Use Macros Safely
- Execution and GitHub Record

Task 1 – Understand the Problem and Identify the Data (4 Marks)

Grama Jyothi Rural Electricity Co-operative calculates each household's electricity bill from meter readings. The program must calculate consumption correctly, apply the tariff slabs, add the fixed charge, and avoid negative or inaccurate bills. It should also be easy to update when the tariff changes. **1.1 Inputs, outputs and why they are needed**

Data item	Input / Output	Reason
Household number	Input	Identifies the household so that the bill is assigned to the correct customer.
Previous meter reading	Input	The reading at the start of the billing period.
Current meter reading	Input	The reading at the end of the billing period.
Customer category	Input (recorded)	Allows a category-specific tariff to be added later. In this version one tariff applies to every category, so the category does not change the bill.
Units consumed	Computed output	Calculated from the two readings, including a meter rollover if one occurs.
Energy charge	Computed output	Calculated from the units consumed and the applicable tariff slabs.
Fixed charge	Computed output	A fixed amount added to the energy charge for each bill.
Total bill	Computed output	The final amount payable: energy charge plus fixed charge.

1.2 Constants and changing values

Constants: slab limits (100 and 200 units), rates (0, 225 and 450 paise per unit), fixed charge (5000 paise) and the meter-register maximum (65535 for a 16-bit register). These are defined in one place, at the top of the program.

Changing values: household number, category, previous reading, current reading, units consumed, energy charge and total bill. These vary by customer or billing period.

1.3 Negative, fractional and large values

Meter readings and units are whole-number counts; a 16-bit unsigned meter register stores readings from 0 to 65535.

A raw subtraction (current − previous) can be negative when a meter rolls over, but valid consumption must not be negative. The rollover rule must therefore be applied.

Money is conceptually expressed in rupees and paise. Money is stored as integer paise to avoid binary floating-point rounding errors.

A district may contain about 800,000 households. The fixed charges alone total 800,000 × 5000 = 4,000,000,000 paise, which is larger than the largest 32-bit signed integer (2,147,483,647). A single bill is at most about 29.4 million paise and fits in 32 bits, but district totals do not, so amounts are stored in `long long`.

1.4 Rules every correct bill must satisfy

- Consumption must be calculated correctly for normal readings and for a meter rollover.
- A valid bill must not be negative.

- Total bill in paise = energy charge in paise + fixed charge in paise.
- The first 100 units have a rate of 0 paise per unit; units 101–200 cost 225 paise each; units above 200 cost 450 paise each.
- The program must use the same tariff constants for every bill so that a rate revision is applied consistently.
- Invalid readings or invalid customer data must be detected instead of silently producing a wrong bill. The program rejects a reading that is not a number or is outside 0 to 65535.

1.5 Assumptions

- The meter is a 16-bit unsigned counter and rolls from 65535 to 0.
- The billing period contains one previous reading and one current reading.
- If the current reading is smaller than the previous reading, the meter is assumed to have rolled over once (rollover rule). A meter that has genuinely gone backwards cannot be distinguished from a rollover by this program and would need a separate meter-fault check.
- The stated tariff is applied to all households unless a category-specific tariff is later supplied.
- The fixed charge is Rs 50.00 per billing period.
- For the initial design, consumption is an integer number of units; smart-meter interval readings can be handled in a later redesign.

1.6 Alternative: store readings as text

A reading could be stored as a character string, for example "01530". The program would need to validate that every character is a digit and convert the string to an integer before calculating consumption. Converting or comparing a reading of d digits takes O(d) time. A string needs approximately d + 1 bytes for d digit characters plus the terminating '\0'; an integer needs a fixed amount of storage, commonly 2 bytes for `unsigned short` on the target system. Text preserves leading zeros and is useful for input validation, but integer storage is simpler for arithmetic. A practical design can read text, validate it, then convert it to an integer.

Conclusion: use integer readings for calculation, integer paise for money, centralised tariff constants, and explicit rollover handling.

1.7 Program (task01_billing.c)

```
#include <stdio.h>

/* Tariff constants - defined in one place */
#define METER_MAX            65535U   /* 16-bit meter register maximum */
#define SLAB1_UNITS          100U
#define SLAB2_UNITS          200U
#define RATE1_PAISE          0LL      /* first 100 units        */
#define RATE2_PAISE          225LL    /* units 101 to 200       */
#define RATE3_PAISE          450LL    /* units above 200        */
#define FIXED_CHARGE_PAISE   5000LL   /* Rs 50.00 per bill      */

int main(void) {     unsigned int household_no,
previous, current, units;     long long energy_charge,
total_bill;

    printf("Enter household number: ");     if
(scanf("%u", &household_no) != 1) {
printf("Error: invalid household number.\n");
return 1;     }

    printf("Enter previous reading: ");     if (scanf("%u", &previous)
!= 1 || previous > METER_MAX) {         printf("Error: previous reading
must be 0 to %u.\n", METER_MAX);         return 1;     }

    printf("Enter current reading: ");     if (scanf("%u",
&current) != 1 || current > METER_MAX) {
        printf("Error: current reading must be 0 to %u.\n",
METER_MAX);         return 1;     }

    /* Units consumed, including meter rollover (modulo 65536) */
units = (current - previous) & METER_MAX;

    /* Energy charge in paise */     if (units <= SLAB1_UNITS)
energy_charge = units * RATE1_PAISE;     else if (units <=
SLAB2_UNITS)         energy_charge = (units - SLAB1_UNITS) *
RATE2_PAISE;     else         energy_charge = (SLAB2_UNITS -
SLAB1_UNITS) * RATE2_PAISE                      + (units -
SLAB2_UNITS) * RATE3_PAISE;

    total_bill = energy_charge + FIXED_CHARGE_PAISE;

    printf("\n--- Electricity Bill ---\n");
printf("Household Number: %u\n", household_no);
printf("Units Consumed: %u\n", units);     printf("Energy
Charge: Rs %lld.%02lld\n",            energy_charge / 100,
energy_charge % 100);     printf("Fixed Charge: Rs
%lld.%02lld\n",            FIXED_CHARGE_PAISE / 100,
FIXED_CHARGE_PAISE % 100);     printf("Total Bill: Rs
%lld.%02lld\n",            total_bill / 100, total_bill %
100);

    return 0;
}
```

Compile: `gcc task01_billing.c -Wall -Wextra -std=c11 -o task01_billing` — no warnings on GCC 13.3.0.

Run 1 – normal reading (1200 to 1530): 330 units = 100 × Rs 0 + 100 × Rs 2.25 + 130 × Rs 4.50 = Rs 225.00 + Rs 585.00 = Rs 810.00; plus Rs 50.00 fixed = Rs 860.00.

```
Enter household number: 101
Enter previous reading: 1200
Enter current reading: 1530

--- Electricity Bill ---
Household Number: 101
Units Consumed: 330
Energy Charge: Rs 810.00
Fixed Charge: Rs 50.00
Total Bill: Rs 860.00
```

Run 2 – meter rollover (65400 to 120): (65536 − 65400) + 120 = 256 units. Energy charge = 100 × Rs 2.25 + 56 × Rs 4.50 = Rs 477.00; total Rs 527.00.

```
Enter household number: 102
Enter previous reading: 65400
Enter current reading: 120

--- Electricity Bill ---
Household Number: 102
Units Consumed: 256
Energy Charge: Rs 477.00
Fixed Charge: Rs 50.00
Total Bill: Rs 527.00
```

Run 3 – within the middle slab (0 to 150): 50 × Rs 2.25 = Rs 112.50; total Rs 162.50.

```
Enter household number: 104
Enter previous reading: 0
Enter current reading: 150

--- Electricity Bill ---
Household Number: 104
Units Consumed: 150
Energy Charge: Rs 112.50
Fixed Charge: Rs 50.00
Total Bill: Rs 162.50
```

Run 4 – invalid reading (70000 is outside the 16-bit range): the program reports an error instead of printing a wrong bill.

```
Enter household number: 103
Enter previous reading: 100
Enter current reading: 70000 Error: current reading
must be 0 to 65535.
```

Task 2 – Choose the Data Types (4 Marks)

2.1 Suggested data types

Data item	Suggested type	Reason / range consideration
Household number	unsigned int	Non-negative identifier; confirm the maximum ID used by the co-operative.
Previous/current reading	meter_t = unsigned short	For a 16-bit meter, range is 0 to 65535.
Units consumed	unsigned int	Non-negative count; can represent rollover consumption up to 65535 for one interval.
Customer category	enum or int	A small set of named categories can be represented clearly with enum.
Tariff rate	long long or int constant	Rates are integer paise per unit; use a type that safely holds the configured rate.
Fixed/energy charge	long long	Integer paise prevents fractional-paise rounding; long long gives ample range on every platform.
Total for district	long long	Fixed charges alone reach 4,000,000,000 paise for 800,000 households, which overflows a 32-bit int.
Temporary text reading	char[]	Used only if reading input as digits; allow space for digits plus '\0'.

2.2 Sizes and ranges

The C standard does not require every implementation to use the same size for int, long or long long. The following are common GCC sizes on a 64-bit Linux system; the measured values from the program below were obtained on the compiler stated on the cover page.

Type	Common size	Range / precision source
Char	1 byte	CHAR_MIN and CHAR_MAX in limits.h; plain char may be signed or unsigned.
unsigned short	2 bytes	0 to USHRT_MAX; commonly 0 to 65535.
Int	4 bytes	INT_MIN to INT_MAX.
unsigned int	4 bytes	0 to UINT_MAX.
Long	8 bytes on many 64-bit Linux systems	LONG_MIN to LONG_MAX; size is platform-dependent (4 bytes on 64-bit Windows).
long long	8 bytes	LLONG_MIN to LLONG_MAX.
Float	4 bytes	FLT_DIG and FLT_EPSILON in float.h; approximately 6–7 decimal digits of precision.
Double	8 bytes	DBL_DIG and DBL_EPSILON in float.h; approximately 15–16 decimal digits of precision.

Useful headers: `<limits.h>` provides integer limits; `<float.h>` provides floating-point precision limits. Use `sizeof(type)` to measure size on the computer where the program is compiled.

Program task02_sizes.c — prints the measured sizes and limits:

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void) {     printf("char           : %zu byte(s)\n", sizeof(char));
printf("unsigned short : %zu byte(s), max = %u\n",             sizeof(unsigned
short), (unsigned)USHRT_MAX);     printf("int            : %zu byte(s), max =
%d\n", sizeof(int), INT_MAX);     printf("unsigned int   : %zu byte(s), max =
%u\n",            sizeof(unsigned int), UINT_MAX);     printf("long           :
%zu byte(s), max = %ld\n", sizeof(long), LONG_MAX);     printf("long long
: %zu byte(s), max = %lld\n",            sizeof(long long), LLONG_MAX);
printf("float          : %zu byte(s), FLT_DIG = %d\n",
sizeof(float), FLT_DIG);     printf("double         : %zu byte(s), DBL_DIG =
%d\n",            sizeof(double), DBL_DIG);     return 0;
}
```

Output:

```
char           : 1 byte(s) unsigned short : 2
byte(s), max = 65535 int            : 4 byte(s), max
= 2147483647 unsigned int   : 4 byte(s), max =
4294967295 long           : 8 byte(s), max =
9223372036854775807 long long      : 8 byte(s), max =
9223372036854775807 float          : 4 byte(s),
FLT_DIG = 6 double         : 8 byte(s), DBL_DIG = 15
```

2.3 Why money should be stored in paise

Binary floating-point cannot represent many decimal fractions exactly. Repeated calculations can therefore produce a value slightly above or below the exact decimal amount. Rs 12.34 is stored as 1234 paise instead. Integer addition is exact as long as the chosen integer type does not overflow. Paise are converted to rupees only when displaying the bill.

Illustration: adding a one-paise bill 4,000 times should give exactly 4,000 paise (Rs 40.00). The float-based repeated sum of 0.01f does not give exactly 40, whereas the integer-paise version does.

Program task02_paise.c

```
#include <stdio.h>

int main(void) {     float
float_total = 0.0f;     long
long total_paise = 0;

    for (int i = 0; i < 4000;
i++)     {         float_total +=
0.01f;         total_paise += 1;
}

    printf("Float total: %.9f rupees\n", float_total);
printf("Integer total: %lld paise = Rs %lld.%02lld\n",
total_paise,            total_paise / 100,
total_paise % 100);

    return 0;
}
```

Output:

```
Float total: 39.999294281 rupees
Integer total: 4000 paise = Rs 40.00
```

2.4 Complete the Code – meter rollover

Program task02_rollover.c

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t
current) {     return (meter_t)(current - previous); }

int main(void) {     printf("sizeof(meter_t) = %zu bytes,
maximum reading = %u\n",            sizeof(meter_t),
(unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
printf("Consumption: %u units\n", consumption(65400, 120));
return 0;
}
```

Output:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

Explanation: for normal readings, 1530 − 1200 = 330. For the rollover case, the counter has 65536 possible values (0 through 65535). The units consumed are (65536 − 65400) + 120 = 136 + 120 = 256. In the code, the expression current − previous is evaluated as a (negative) int, and the conversion to `unsigned short` wraps it modulo 65536, so 120 − 65400 = −65280 becomes −65280 + 65536 = 256, not a negative number. This assumes the meter has not completed more than one full rollover between readings.

2.5 Alternative: double rupees

If money were stored as double rupees, each charge could be calculated in rupees and rounded to two decimal places when the bill is finalised or displayed. However, rounding only at the end can produce a different result from rounding each bill to paise first. A double addition and an integer addition are both O(1) time per operation. A double commonly uses 8 bytes, and a 64-bit integer also uses 8 bytes. Floating-point rounding error can accumulate as n values are added; integer paise has no rounding error for additions, provided overflow is prevented.

Task 3 – Define the Constants

Tariff limits, rates and the fixed charge should be defined in one place. This prevents different parts of the program from using inconsistent rates after a tariff revision.

3.1 #define, const and enum

Feature	Best use	Important difference
#define	Simple compile-time numeric macros such as slab limits and rates.	Preprocessor text replacement; no type checking for the replacement itself and no variable storage.
const variable	Typed value that should not be modified.	Has a type and follows normal scope rules; can be easier to inspect in a debugger, though storage is not always required.
enum	Named integral choices such as customer categories or meter states.	Improves readability for a fixed set of integer labels; not intended for fractional tariff amounts.

Use uppercase names with underscores for macros, such as SLAB1_UNITS and FIXED_CHARGE_PAISE. Keep units in the name (PAISE, UNITS) to reduce confusion. Short examples of the other two forms:

```
const long long FIXED_CHARGE = 5000LL;   /* typed constant, normal scope */

enum customer_category {               /* named categories: 0, 1, 2 */
CAT_DOMESTIC,
    CAT_COMMERCIAL,
    CAT_AGRICULTURE
};
```

3.2 Complete C program (task03_constants.c)

```
#include <stdio.h>

#define SLAB1_UNITS        100
#define SLAB2_UNITS        200
#define RATE1_PAISE        0
#define RATE2_PAISE        225
#define RATE3_PAISE        450
#define FIXED_CHARGE_PAISE 5000LL

/* Returns the bill in paise */ long long
compute_bill(unsigned int units) {
long long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {         paise += (long
long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;         paise +=
(long long)(units - SLAB2_UNITS) * RATE3_PAISE;     }     else if
(units > SLAB1_UNITS) {         paise += (long long)(units -
SLAB1_UNITS) * RATE2_PAISE;     }     else {         paise += (long
long)units * RATE1_PAISE;     }     return paise; }

int main(void) {     unsigned int tests[] = {0,
100, 150, 200, 350};     int n = sizeof tests /
sizeof tests[0];

    for (int i = 0; i < n; i++) {         long long p =
compute_bill(tests[i]);         printf("%3u units -> Rs %lld.%02lld\n",
tests[i], p / 100, p % 100);
    }
return 0; }
```

Compile: `gcc task03_constants.c -Wall -Wextra -std=c11 -o task03_constants` — no warnings.

Output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Check by hand: 150 units = Rs 50 + 50 × Rs 2.25 = Rs 162.50; 200 units = Rs 50 + 100 × Rs 2.25 = Rs 275.00; 350 units = Rs 50 + Rs 225 + 150 × Rs 4.50 = Rs 950.00.

3.3 Checking a tariff revision

Change `RATE2_PAISE` from 225 to 230 and compile the same program again. The 150-unit bill becomes Rs 165.00 (Rs 50 fixed + 50 × Rs 2.30), and the 200-unit bill becomes Rs 280.00. The 0- and 100-unit bills remain Rs 50.00. The 350-unit bill becomes Rs 955.00 because the middle slab increases by Rs 5.00 (100 × 5 paise); the above-200 slab rate remains unchanged.

Output with RATE2_PAISE = 230:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

3.4 Alternative concept and complexity

If #define values were replaced by const variables, the program would gain typed constants and normal scope rules. If enum were used, it would suit named categories rather than rates such as 225 paise. A #define normally uses no storage; a const object may use storage depending on how it is used and compiled. These choices do not change the billing algorithm’s asymptotic running time: calculating one bill remains O(1) time and O(1) extra space.

3.5 Conclusion

Using well-named constants prevents tariff values from being duplicated throughout the billing program. #define is suitable for these fixed tariff values, while const provides typed values and enum is useful for categories. Testing a tariff revision confirms that the change is applied consistently.

Task 4 – Use Macros Safely

A macro is expanded by the C preprocessor before compilation. Parentheses are important because the replacement text is inserted into the program without ordinary function-call type checking.

4.1 Introduction

Macros are defined using #define. Before the C program is compiled, the preprocessor replaces each macro with its defined expression.

Macros can reduce repeated code, but they must be written carefully. Missing parentheses can cause incorrect calculations, and some macros can evaluate their arguments more than once. **4.2 C program (task04_macros.c)**

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((long)(r) * 100L)
#define BAD_SQUARE(x) x * x
#define SQUARE(x) ((x) * (x)) #define
MAX(a, b) (((a) > (b)) ? (a) : (b))

int
main(void) {
int a = 3;
int i = 6;
int j = 5;

    printf("12 rupees in paise = %ld\n",
RUPEES_TO_PAISE(12));     printf("Bad square of a + 1 =
%d\n", BAD_SQUARE(a + 1));     printf("Safe square of a + 1 =
%d\n", SQUARE(a + 1));     printf("Maximum of 10 and 20 =
%d\n", MAX(10, 20));     printf("Before MAX: i = %d, j =
%d\n", i, j);     printf("MAX(i++, j) = %d\n", MAX(i++, j));
printf("After MAX: i = %d, j = %d\n", i, j);

    return 0;
}
```

Compile: `gcc task04_macros.c -Wall -Wextra -std=c11 -o task04_macros` — no warnings.

Output:

```
12 rupees in paise = 1200
Bad square of a + 1 = 7
Safe square of a + 1 = 16
Maximum of 10 and 20 = 20
Before MAX: i = 6, j = 5
MAX(i++, j) = 7
After MAX: i = 8, j = 5
```

Explanation of the results:

- `BAD_SQUARE(a + 1)` expands to `a + 1 * a + 1`. Multiplication happens first, so the value is 3 + 3 + 1 = 7, not 16.
- `SQUARE(a + 1)` expands to `((a + 1) * (a + 1))` = 16.
- `MAX(i++, j)` expands to `(((i++) > (j)) ? (i++) : (j))`. The comparison uses i = 6 and then increments i to 7; because 6 > 5 is true, i++ is evaluated a second time, so the result is 7 and i ends at 8, although the call looks as if it should increment i only once.

4.3 Macro pitfalls

Double evaluation: in MAX(i++, j), i++ is evaluated once in the comparison and again in the selected expression, so i is incremented twice (see the output above). Avoid side effects in macro arguments.

Type safety: macros perform text substitution and do not create a typed function interface. Unexpected argument types may lead to confusing results.

Debugging: a macro may be harder to step through in a debugger because it is expanded before compilation.

Use a function when an expression has side effects, when type checking is important, or when the calculation is more than a simple expression.

4.4 Show preprocessor expansion with gcc -E

Save a C source file containing the macro definitions and a use such as `int area = SQUARE(a + 1);`.

```
#define SQUARE(x) ((x) * (x))

int main(void) {     int a = 3;     int area = SQUARE(a
+ 1);     return area; }
```

Run:

```
gcc -E macros.c -o macros.i
grep -n "int area" macros.i
```

Output (line 12 of macros.i on the test system):

```
12:    int area = ((a + 1) * (a + 1));
```

The preprocessor has replaced SQUARE(a + 1) with the fully parenthesised expression. The actual preprocessed file also contains line markers, so its full output is longer; the line number depends on the system headers. Keep a screenshot of the command and the relevant expanded line as execution evidence.

4.5 Alternative: static inline functions

```
static inline long rupees_to_paise(long
rupees) {     return rupees * 100L; }

static inline int square(int
x) {     return x * x; }

static inline int maximum(int a, int
b) {     return (a > b) ? a : b;
}
```

An inline function has typed parameters and evaluates each argument once. The compiler may inline it, so there may be no call overhead; if it is not inlined, a function call may add a small constant cost. Both macro expansion and a simple inline function perform O(1) work for these calculations. If a macro is used k times, its replacement code is repeated at each use and can increase code size roughly with k. An inline function can allow the compiler to share one function body or inline it selectively. Actual code size depends on compiler optimisation.

Output of task04_inline.c (same tests as above) — note that i is incremented only once:

```
12 rupees in paise = 1200
Square of 3 + 1 = 16
maximum(i++, j) = 6 After
maximum: i = 7, j = 5
```

4.6 Summary

Use fully parenthesised macros only for simple, safe expressions. Do not pass arguments with side effects to macros such as MAX. For type safety and predictable argument evaluation, static inline functions are generally preferable.

Execution and GitHub Record

Repository name: **CSA02 Assignment1** (REG NO :1924260209)

Folder	Source file(s)	Compile command
Task01	task01_billing.c	`gcc task01_billing.c -Wall -Wextra -std=c11 -o task01_billing`
Task02	task02_sizes.c task02_paise.c task02_rollover.c	`gcc task02_sizes.c -Wall -Wextra -std=c11 -o task02_sizes gcc task02_paise.c -Wall -Wextra -std=c11 -o task02_paise gcc task02_rollover.c -Wall -Wextra -std=c11 -o task02_rollover`
Task03	task03_constants.c	`gcc task03_constants.c -Wall -Wextra -std=c11 -o task03_constants`
Task04	task04_macros.c macros.c task04_inline.c	`gcc task04_macros.c -Wall -Wextra -std=c11 -o task04_macros gcc -E macros.c -o macros.i gcc task04_inline.c -Wall -Wextra -std=c11 -o task04_inline`